

어드민 페이지 토큰 해제 및 새로고침 이슈 원인 분석

문제 상황:

`admin.standardai.co.kr` 어드민 페이지에서 “토큰 해제” 버튼 (`clearAdminTokenJJ()`)과 “새로고침” 버튼 (`refresh()`)이 정상 작동하지 않는 문제가 발생했습니다. 구체적으로, 토큰을 제거하는 버튼을 눌러도 로컬 스토리지에 저장된 토큰이 사라지지 않거나 페이지가 제대로 갱신되지 않고, KPI 수치를 갱신하는 새로고침 버튼을 눌러도 `/api/status` 호출에 의한 UI 업데이트가 이루어지지 않습니다. 또한 초기 목록 갱신 함수 (`loadReady()`)가 호출되지 않아 `/api/items?state=ready`를 통한 데이터 갱신이 이뤄지지 않는 현상도 함께 보고되었습니다. 아래에서는 이 문제의 근본 원인을 여러 각도에서 분석하고, 코드 수준의 수정 방안을 제시합니다.

동작 환경 및 전제 조건

- **프론트엔드 구조:** Vue나 React 등의 프레임워크 없이 순수 **HTML + JavaScript**로 작성되어 있습니다. UI 상의 버튼에 직접 이벤트를 바인딩하거나, 전역 함수로 onclick을 지정하는 방식으로 동작합니다.
- **인증 토큰 관리:** 인증용 토큰(예: JWT)은 **localStorage**에 저장되며, 커스텀 **window.fetch** 오버라이드 함수를 통해 모든 API 호출 시 자동으로 **Authorization** 헤더에 토큰을 포함하도록 구현되었습니다.
- **토큰 해제 기능:** “토큰 해제” 버튼 (`id="btn-token-clear"`)을 누르면 `clearAdminTokenJJ()` 함수를 호출하여 localStorage에 저장된 토큰(예: 키 이름 `ADMIN_TOKEN`)을 삭제하고 페이지를 새로고침하도록 되어 있습니다.
- **새로고침(KPI 갱신) 기능:** “새로고침” 버튼을 누르면 `refresh()` 함수를 통해 `/api/status` 엔드포인트를 호출, 응답으로 받은 KPI 지표 데이터를 화면에 갱신해야 합니다.
- **목록 갱신 기능:** 별도의 `loadReady()` 함수가 있어 `/api/items?state=ready` 엔드포인트를 호출함으로써 특정 상태(`ready`)의 아이템 목록을 가져와 표시하게 되어 있으나, 현재 이 함수가 호출되지 않거나 응답 처리에 문제가 있어 목록이 갱신되지 않는 것으로 보입니다.

발생한 문제와 증상

1. **토큰 해제 버튼 무반응:** `btn-token-clear` 버튼 클릭 시 아무 일도 일어나지 않음. 예상대로라면 localStorage의 토큰이 제거되고 페이지가 리로드되어야 하나, 실제로는 토큰이 그대로 남아 있거나 페이지가 갱신되지 않습니다. 사용자는 로그아웃이 되지 않아 이전 인증 상태가 지속되는 문제를 겪습니다.
2. **새로고침(KPI) 버튼 무반응:** KPI 수치를 갱신하는 새로고침 버튼 클릭 시 화면에 아무 변화가 없음. 정상 동작이라면 `/api/status`를 호출한 뒤 반환된 수치들로 UI를 업데이트해야 하나, **Network 탭에 해당 API 요청이 보이지 않거나** UI 숫자가 갱신되지 않습니다.
3. **목록 자동 갱신 실패:** 페이지 로드 후 자동으로 호출되어야 할 `loadReady()`가 실행되지 않거나 실행되더라도 **UI에 반영이 되지 않음**. 이로 인해 특정 상태의 아이템 리스트가 초기화 또는 업데이트되지 않고 남아 있습니다 (예: “Ready” 상태 아이템 목록이 비어있거나 오래된 데이터인 채로 남음).

다음 섹션에서는 이러한 현상의 근본 원인을 의심할 수 있는 항목들을 분석합니다.

근본 원인 분석

1. UI 이벤트 바인딩 문제

설명: 토큰 해제 및 새로고침 버튼의 **클릭 이벤트가 자바스크립트 함수에 연결되지 않은 경우**, 버튼을 눌러도 아무 일도 일어나지 않습니다. 가능한 원인은 다음과 같습니다:

- **스크립트 로드/실행 시점 오류:** 이벤트 바인딩 스크립트가 DOM 요소보다 먼저 실행된 경우입니다. 예를 들어 `<script>`를 `<head>`에 배치하고 `DOMContentLoaded` 이벤트를 기다리지 않았다면, `document.getElementById('btn-token-clear')` 등이 null을 반환하여 이벤트 핸들러가 등록되지 않았을 수 있습니다. 이 경우 버튼 클릭에 대한 반응이 전혀 없게 됩니다.
- **요소 ID 혹은 함수 이름 불일치:** HTML 버튼의 `id`나 `onclick` 속성 명칭이 스크립트에서 사용하는 이름과 다르면 이벤트가 연결되지 않습니다. (예: 버튼의 id가 `btn-token-clear`인데 스크립트에서는 `btn_clear_token`으로 찾고 있다면 매칭 실패)
- **스크립트 파일 누락 또는 오류:** 프론트엔드 배포 시 해당 기능을 담당하는 JS 파일이 누락되어 로드되지 않거나, 스크립트 자체에 **런타임 오류**가 있어 초기화 코드가 중단되었을 수 있습니다. 배포 환경에서 **정적 리소스 파일이 빠졌거나 캐시된 구버전이 로드되었다면**, 이벤트 바인딩 코드가 아예 실행되지 않아 이런 증상이 나타날 수 있습니다 ① ②.

DevTools 확인: Chrome 개발자 도구의 **Console 탭**에서 페이지 로드 시 에러 메시지가 있는지 먼저 확인합니다 ③. 만약 스크립트 파싱 에러나 `Uncaught ReferenceError` 등이 있다면, 해당 오류가 이벤트 바인딩 코드를 건너뛰게 했을 가능성이 큼니다. 또한 **Elements 탭**에서 문제가 된 버튼 요소를 선택해 **Properties 혹은 Event Listeners**를 확인하면, click 이벤트 리스너가 연결되어 있는지 검사할 수 있습니다. `document.getElementById('btn-token-clear')`의 반환값이 콘솔에서 null이 아닌지 직접 확인해 보는 것도 도움이 됩니다.

네트워크 탭에서도 버튼 클릭 시 **/api/status 호출이 발생하지 않는지** 확인해보세요 ③. 호출 자체가 없으면 이벤트 핸들러 미바인딩이 확실하며, 호출 시도는 있으나 응답이 없거나 에러가 난다면 다른 원인을 추가로 살펴봐야 합니다.

2. window.fetch 오버라이드로 인한 부작용

설명: 이 프로젝트에서는 `window.fetch`를 커스터마이징하여 토큰을 자동으로 헤더에 실어 보내도록 하고 있습니다. 잘못된 fetch 오버라이드 구현은 **API 호출 누락이나 자바스크립트 오류**를 일으킬 수 있습니다:

- **재귀 호출 버그:** `window.fetch`를 재정의할 때 원본 fetch를 잘 보관하지 않고 `window.fetch()`를 다시 호출하면 **무한 재귀**가 발생합니다. 올바른 방법은 예를 들어 `const originalFetch = window.fetch; window.fetch = function(...){ ...; return originalFetch(...); }`처럼 원본 참조를 사용해야 합니다 ④ ⑤. 만약 이 부분이 잘못되어 있다면 fetch 호출 시 스크립트 오류가 발생해 이후 코드 (예: UI 업데이트) 실행이 중단되었을 것입니다.
- **헤더 설정 로직 오류:** fetch 재정의 시 `init.headers` 처리에 버그가 있을 경우입니다. 예를 들어 `init.headers`가 `Headers` 객체일 때와 일반 객체일 때를 구분하지 않으면 헤더 추가 시 에러가 날 수 있습니다 ⑤. 또한 `init` 파라미터가 `undefined`인 경우를 대비하지 않았다면 `fetch('/api/status')` 호출 시 `init.headers` 접근에서 오류가 발생할 수 있습니다. 이런 오류 역시 Console에서 확인 가능하며, 존재한다면 fetch 호출이 아예 진행되지 않아 새로고침 버튼이 무반응으로 보였을 것입니다.
- **토큰 미존재 시 처리:** 토큰이 없을 때 fetch를 호출하면, Authorization 헤더를 생략하거나 기본 동작을 해야 합니다. 그런데 오버라이드 구현이 “토큰이 없으면 fetch를 호출하지 않도록” 잘못 짜인 경우 (예: 토큰 없으면 아무 것도 안 하고 return 한다면) 토큰이 지워진 후에는 fetch가 작동하지 않을 것입니다. 이 경우 토큰 해제 후 `/api/status`를 호출해 상태를 검증하는 로직 등이 실행되지 않겠지요 ⑥. 따라서 토큰이 없는 상황에서도 fetch 오버라이드 함수가 **원본 fetch를 정상적으로 호출하도록** 구현되어 있는지 확인이 필요합니다.

DevTools 확인: Console 탭에서 fetch와 관련한 오류 메시지를 찾습니다. 특히 **버튼 클릭 시** “Maximum call stack size exceeded” (재귀 호출)이나 “Cannot read property ‘headers’ of undefined” 등의 오류가 없는지 살펴보세요. 또한 Network 탭에서 `/api/status` 나 `/api/items` 요청이 아예 보이지 않으면, fetch 재정의 로직이 해당 호출을 가로채고 있지 않은지 의심해야 합니다. 필요하다면 브레이크포인트를 걸어 `window.fetch` 함수 내부 동작을 트레이싱해보는 것도 방법입니다.

3. 토큰 삭제 및 페이지 새로고침 로직

설명: `clearAdminTokenJJ()` 함수의 구현 방식에도 문제가 있을 수 있습니다. 일반적으로 로그아웃 기능은 **토큰 제거 → 상태 초기화/확인 → 페이지 리로드 또는 로그인 페이지로 리디렉션** 순으로 이뤄집니다. 현 상황에서 고려할 만한 원인은:

- **이벤트 핸들러 내 즉시 리로드:** 현재 추정으로는 `clearAdminTokenJJ()` 에서 `localStorage.removeItem('ADMIN_TOKEN')` 후 `location.reload()` 를 바로 호출하는데, 이 과정에서 **이벤트 핸들러가 완전히 수행되기 전에 페이지 전환**이 일어나 버릴 가능성이 있습니다. 원칙적으로 `removeItem`은 동기적으로 실행되므로 토큰은 삭제되지만, `reload`로 인해 이후 이어질 다른 코드 (예: 상태 체크 호출)가 실행되지 않을 수 있습니다 ⑥. 예를 들어 토큰 삭제 후 곧바로 `/api/status` 를 호출해 **백엔드가 401을 반환하는지 확인하고 UI에 반영**하려는 의도가 있었다면, 즉시 `reload`해버리면 그 검증이 이루어지지 않습니다.
- **캐시된 페이지 재로딩 문제:** “토큰 제거 직후 페이지가 새로고침되면서 캐시된 HTML이 다시 토큰을 넣어준 다”는 가설도 언급되었습니다 ⑥. 이는 예전 버전의 스크립트나 HTML이 브라우저 캐시에 남아 있어, `reload` 시 최신 코드 대신 이전 코드가 실행되면서 이전 토큰을 다시 설정하거나, 로그아웃이 반영되지 않는 현상을 의미합니다. 만약 과거 코드에 `localStorage.setItem('ADMIN_TOKEN', ...)` 하는 부분이 있었다면 (예: 쿠키나 특정 저장소로부터 토큰을 재로딩하는 로직), 캐시로 인해 그 부분이 여전히 동작해 토큰이復活할 수 있습니다.
- **토큰 중복 저장 경로:** 혹시 토큰이 여러 곳에 저장되고 있는지도 확인해야 합니다. 예를 들어 `localStorage` 외에 쿠키에도 토큰이나 세션이 존재한다면, 페이지 리로드 시 쿠키에 있는 토큰으로 다시 `localStorage`를 채우는 코드가 있을 수 있습니다. 이럴 경우 `removeItem` 만으로는 완전한 로그아웃이 안 되고, UI에서 토큰이 “안 지워진 것처럼” 보이게 됩니다.

DevTools 확인: Application 탭의 **Local Storage** 섹션을 열어 `ADMIN_TOKEN` 값이 버튼 클릭 전후로 어떻게 변화하는지 관찰하세요. 클릭 후에도 `localStorage`에 남아 있다면 event가 아예 동작하지 않은 것이고, 일시적으로 지워졌다가 `reload` 후 다시 나타난다면 **reload 과정에서 토큰이 복구되고 있을** 가능성이 있습니다. 이때 **Console 탭** 로그를 보면 `reload` 이전에 어떤 스크립트가 실행되었는지, **Network 탭**을 통해 새로고침 요청이 어느 리소스를 불러왔는지 (캐시 여부, 응답 코드 등) 확인할 수 있습니다. 만약 `reload` 후 HTML/JS가 **갱신되지 않은 타임스탬프**를 가진다면, 강력 새로고침(캐시 비우고 불러오기)을 시도해 보고, 문제 재현 여부를 비교해 캐싱 영향을 판단해야 합니다.

4. `loadReady()` 함수 미호출 또는 무반응

설명: `loadReady()` 는 준비(ready) 상태의 아이템 목록을 불러오는 함수인데, 현재 **페이지 로드 시 해당 함수가 호출되지 않거나** 호출되어도 화면에 변화가 없는 상황입니다. 이 부분의 원인으로는:

- **초기화 로직 누락:** 페이지 로딩 후 자동으로 `loadReady()` 를 호출해주어야 하는데, 그 호출 구문이 누락되었을 수 있습니다. 예를 들어 초기화 스크립트에서 `refresh(); loadReady();` 순서로 부르려 했는데 실수로 `loadReady();` 를 빠뜨렸다면, KPI는 갱신되어도 목록은 갱신되지 않을 것입니다.
- **이벤트 트리거 누락:** 혹은 UI 상의 다른 요소 (예: 탭 클릭 등)에 `loadReady()` 호출이 바인딩되어 있어야 하는데 빠졌을 가능성도 있습니다. 이 경우 사용자가 해당 기능을 전혀 실행할 수 없습니다.
- **백엔드 응답 처리 버그:** `loadReady()` 내부 구현에서 fetch 결과를 제대로 파싱하지 못하거나, DOM 업데이트 부분에서 오류가 발생하면 데이터를 받아와도 화면에 표시되지 않을 수 있습니다. 만약 fetch 호출은 실행되지만 `.then` 또는 DOM 조작에서 JS 에러가 난다면, Console에 오류가 남고 UI 업데이트가 중단됩니다.

DevTools 확인: Network 탭을 보고 **페이지가 처음 로드될 때** `/api/items?state=ready` 요청이 있는지 찾습니다. 없다면 초기 호출 누락이 의심되므로 코드 상 해당 부분을 찾아봐야 합니다. 요청은 있지만 응답이 200이고도 화면에 변화가 없다면, **Console 탭**에 그 응답을 처리하는 부분에서 에러가 없는지 확인합니다. 또한, `loadReady()` 함수를 수동으로 Console에서 호출해 보아 실제 네트워크 요청이 발생하는지, 그리고 발생한 데이터를 가지고 DOM을 업데이트하는지 한 번 실험해볼 수도 있습니다. 이를 통해 함수 자체는 유효하지만 호출 타이밍이나 바인딩 문제임을 좁혀나갈 수 있습니다.

5. 백엔드 API 연동 및 기타 가능성

설명: 프론트엔드 문제가 아니라 **백엔드/API 응답이나 경로 설정 문제**일 가능성도 배제할 수 없습니다. 몇 가지 점검사항:

- **API 엔드포인트 경로 확인:** 프론트엔드에서 호출하는 경로가 `/api/...`로 시작하는데, 실제 배포 환경에서 이 경로가 올바르게 Cloud Run 서비스로 연결되는지 확인해야 합니다 ⁷. 만약 프론트엔드와 백엔드가 다른 도메인이라면 CORS나 경로 prefix 문제로 호출이 실패할 수 있고, 같은 도메인/서비스라 해도 Cloud Run 리비전이나 도메인 매핑 문제로 `/api` 요청이 404 또는 다른 서비스로 가는 문제가 이전에 있었습니다. (실제로 이번 사례에서도 **도메인이 구버전 서비스를 가리키던 것을 교정했다고 언급**되었습니다 ⁸.)
- **API 인증 응답 확인:** 백엔드 `/api/status` 나 `/api/items` 등이 토큰 존재 시 200 OK, 없으면 401/403을 정확히 주는지 **별도로 확인**할 필요가 있습니다 ⁷. 만약 백엔드가 올바르게 동작하는데도 UI에서 해당 호출이 보이지 않거나 실패한다면, **Authorization 헤더 전송 여부(fetch 오버라이드)**나 요청 방식 등에 문제가 있을 수 있습니다.
- **정적 파일 캐시/배포:** 앞서 언급한 대로 HTML/JS 파일이 최신 버전으로 배포되었는지와 브라우저 캐시가 아닌지 확인해야 합니다. Cloud Run 등의 정적 콘텐츠 배포 시 적절한 캐시 무효화 정책이나 파일 명시적 버전 관리가 없으면, 클라이언트가 이전에 받은 스크립트를 계속 사용할 수 있습니다. 특히 **토큰 해제 및 새로고침 관련 기능은 최근에 수정된 것**이라면, 구버전 파일을 사용하는 환경에서만 문제가 재현될 수 있으므로, 배포 파이프라인이 해당 파일들을 포함하고 최신으로 갱신했는지 점검이 필요합니다 ⁹.

DevTools 확인: Network 탭에서 `/api/...` 호출의 **HTTP 상태 코드와 응답**을 확인하세요. 백엔드가 401 등을 주고 있다면 프론트엔드가 이를 제대로 처리 못 하는 로직 문제일 수 있고, 요청 자체가 아예 가지 않았다면 앞서 언급한 이벤트/스크립트 이슈입니다. 또한 정적 자원 (HTML, JS)의 응답 헤더에 `Cache-Control` 등이 어떻게 설정되었는지 보면서, 필요하다면 **강제 새로고침**(Ctrl+F5 등)으로 최신 스크립트를 불러오는지 확인합니다.

코드 수준의 해결책 제안

위 원인들을 토대로, 다음과 같은 최소한의 코드 수정으로 문제를 해결할 수 있습니다.

1. 이벤트 바인딩 수정:

- **DOM 로딩 후 이벤트 연결:** 스크립트가 `<head>`에 있다면 `defer` 속성을 추가하거나, `<body>` 끝부분으로 옮겨 **DOM 생성 후 실행**되도록 합니다. 또는 `window.addEventListener('DOMContentLoaded', initFunc)` 형태로 초기화 함수를 등록해 버튼 요소들이 존재한 뒤 이벤트를 바인딩하게 수정합니다. 이렇게 하면 `btn-token-clear` 나 새로고침 버튼에 대한 `addEventListener('click', ...)` 호출이 정상적으로 대상 요소를 찾아 연결할 것입니다. - **ID/함수명 일치 검사:** HTML과 JS 코드에서 사용하는 ID와 함수 이름을 통일합니다. 예를 들어 HTML 버튼에 `onclick="clearAdminTokenJJ()"`로 되어 있다면 JS에도 동일한 함수명이 존재해야 하고, `document.getElementById('btn-token-clear')`로 이벤트를 걸 예정이면 해당 ID가 정확히 HTML에 있어야 합니다. 오타나 불일치가 발견되면 즉시 수정합니다. - **버튼 타입 지정:** 혹시 `<button>` 요소를 사용중이고 폼 내에 있다면 `type="button"`을 명시하여, 클릭 시 불필요한 폼 submit으로 페이지가 리로드되지 않도록 합니다. 이는 이벤트 핸들러를 타기도 전에 폼 제출로 인해 페이지가 새로고침되는 사소한 문제를 예방합니다.

2. `fetch` 오버라이드 코드 수정:

- **원본 fetch 참조 사용:** 재정의한 `window.fetch` 함수 내부에서 반드시 원래의 `fetch` 함수를 호출하도록 수정합니

다.

예시:

```
const _origFetch = window.fetch; window.fetch = function(url, options){ /* 헤더 추가 로직 */ return _origFetch(url, options); }.
```

이렇게 하면 재귀 호출을 피하면서 헤더 삽입이 가능합니다 ⁴ ⁵. 현재 코드에 `_origFetch` 같은 변수가 없다면 이를 추가하고, `window.fetch` 재정의 직전에 저장하도록 합니다. - **옵션 및 헤더 처리 보강:** `options` (또는 `init`) 파라미터가 `undefined` 일 때 객체로 초기화하고, `options.headers` 가 없다면 새 객체 또는 `Headers` 인스턴스로 초기화합니다. 또한 `options.headers` 가 이미 `Headers` 객체일 경우 `append` 메서드로, 일반 객체면 `options.headers['Authorization'] = 'Bearer ' + token` 식으로 올바르게 토큰 헤더를 추가합니다. 이러한 분기 처리를 추가하여 어떤 형태로 `fetch`가 호출되든 `Authorization` 헤더 설정이 빠지지 않고, 오류가 발생하지 않게 합니다. - **예외 및 토큰 없음 처리:** 토큰이 존재하지 않는 경우에도 `window.fetch` 는 원본 `fetch`를 그대로 호출하도록 합니다. 예를 들어 토큰이 없으면 헤더 추가를 생략하고 `return _origFetch(url, options);` 만 수행하면 충분합니다. 절대로 토큰 없다고 `fetch` 호출을 막거나 `null` 을 리턴해서는 안 됩니다. 이렇게 해야 토큰이 없는 상태에서 호출하는 `/api/status` 등이 정상적으로 시도되어, 401 오류를 감지하거나 사용자에게 비로그인 상태를 보여줄 수 있습니다 ⁶.

3. 토큰 해제 후 처리 로직 개선:

- **리로드 이전에 상태 반영:** `clearAdminTokenJJ()` 구현을 검토하여, **토큰 삭제 직후 즉시 `location.reload()` 를 호출하는 부분을 조정**합니다. 최소 수정으로는, `location.reload()` 호출을 **토큰 삭제 후 약간 지연**시키는 방법이 있습니다. 예를 들어 `localStorage.removeItem('ADMIN_TOKEN');` `setTimeout(() => location.reload(), 0);` 처럼 호출하면, 현재 호출 스택을 빠져나간 뒤 `reload`가 일어나 이벤트 핸들러 내 나머지 로직 (필요하다면 `/api/status` 확인 등)을 수행할 시간을 줄 수 있습니다. 더 나아가 가능하다면, 아예 `location.reload()` 를 제거하고 **토큰 삭제 후 `refresh()` 함수**를 호출하여 KPI 상태를 갱신(이 경우 인증 실패로 401 응답 예상)하고, UI에 “로그아웃됨”을 표시하거나 로그인 페이지로 안내하는 흐름으로 개선하는 것이 바람직합니다 ⁶. 이렇게 하면 **토큰 해제 버튼 클릭 시 즉각적인 효과**: “인증 정보가 삭제되었습니다” 라는 UI 변화나 `redirect`를 확인할 수 있고, 필요 시 이후 수동 새로고침을 통해서도 이전 세션이 복구되지 않습니다. - **토큰 복원 경로 차단:** 과거 코드나 다른 저장소로부터 토큰이 다시 설정되는 로직이 없는지 확인합니다. 만약 있다면 (예: 쿠키에 토큰이 남아있는 경우, 로그아웃 시 해당 쿠키도 지우거나, 쿠키를 읽어 `localStorage`에 설정하는 코드를 제거해야 함), **로그아웃 동작 시 모든 저장 경로의 토큰이 삭제**되도록 처리해야 합니다. 이를 통해 `reload` 후에도 토큰이 부활하는 일을 원천 차단합니다. - **캐시 무효화:** 배포된 정적 파일이 갱신되지 않아 구버전 스크립트가 남용되는 경우, **파일 이름에 버전 해시를 넣거나 `Cache-Control` 헤더를 짧게** 설정하여 항상 최신 스크립트를 로드하게끔 수정합니다. 근본적으로 배포 파이프라인에서 HTML/JS가 최신 코드로 교체되었는지 확인하고, 필요 시 배포 환경 캐시(`clear cache`) 등의 조치를 취합니다.

4. `loadReady()` 호출 및 UI 업데이트 수정:

- **초기 호출 추가:** 만약 코드 상에 `loadReady()` 호출이 누락된 것이 확인된다면, **페이지 로드 완료 시점**(예: 초기화 함수 말미나 `window.onload`) 또는 관련 UI 이벤트에서 `loadReady()` 를 호출하도록 코드를 추가합니다. 이 한 줄 추가만으로도 기본적인 목록 갱신 기능이 복구될 수 있습니다. - **DOM 업데이트 로직 검토:** `loadReady()` 함수 내부에서 DOM을 업데이트하는 부분에 오류가 있을 수 있으므로, 해당 부분의 최소 수정도 검토합니다. 예를 들어 응답 데이터를 파싱한 후 `.innerText` 나 `.appendChild` 등을 호출할 때 대상 요소의 selector가 잘못되거나, 예상치 못한 데이터 형식으로 인해 에러가 난다면 `try-catch`로 감싼다든지, 또는 조건문으로 해당 엘리먼트 존재 여부를 체크하여 에러를 방지합니다. **오류가 발생하지 않도록 최소한의 방어 코드**를 넣으면 나머지 실행이 중단되지 않고 UI 갱신까지 이어지게 할 수 있습니다. - **중복 호출/상태 관리:** 혹시 `refresh()` 함수 내에서도 `loadReady()` 를 호출해야 논리상 맞다면, 그 호출이 빠졌는지 확인하고 필요하면 추가합니다. 예컨대 KPI 새로고침 후 목록도 새 상태로 동기화해야 한다면 `refresh()` 끝부분에 `loadReady()` 를 호출해주는 식으로 연계합니다. 이 수정 역시 한 줄 추가로 큰 부작용 없이 개선할 수 있습니다.

5. 기타 환경 설정 조정:

- **API 경로 및 헤더 확인:** 프론트엔드 스크립트에서 `fetch` 호출 시 **절대경로 또는 현재 도메인 기준으로 정확히 API를 호출**하는지 재검토하세요. Cloud Run 등의 환경에서는 경로가 `/api/...` 그대로 통하기도 하지만, 필요에 따라 전

체 URL 또는 적절한 프록시 설정이 되어야 합니다. 만약 경로 문제로 호출이 안 된다면, 자바스크립트에서 base URL을 조정하거나 Cloud Run 설정을 변경하는 등의 조치가 필요합니다 7. - **백엔드 예외 처리**: 백엔드에서 401이 내려온 경우 프론트엔드 fetch Promise가 reject될 것이므로, `.catch`를 통해 오류를 핸들링하는 코드를 추가하는 것이 좋습니다. 예를 들어 `refresh()` 구현 내 `fetch` 호출 뒤 `.catch(err => { console.error(err); /* 사용자에게 로그아웃 상태 표시 등 */ })`를 붙여두면, 새로고침 시 인증오류가 발생해도 조용히 실패하는 대신 콘솔에 찍히고 필요한 UI처리를 할 수 있습니다. 이는 기능 개선 측면이지만 디버깅에도 유용합니다.

以上와 같은 최소한의 수정으로도 현재 드러난 문제들은 해결될 가능성이 높습니다. 핵심은 **이벤트가 올바르게 바인딩** 되고, **fetch 오버라이드가 정상 동작**하며, **토큰 삭제 후의 흐름이 논리적으로** 이어지도록 만드는 것입니다.

브라우저 개발자 도구 활용 확인 항목

문제를 수정한 후에도, 그리고 수정 전 원인 파악 단계에서도 Chrome 개발자 도구를 적극 활용해야 합니다. 다음은 각 문제별로 개발자 도구에서 확인해야 할 사항들입니다:

- **콘솔 오류 확인**: 페이지 로드 시와 버튼 클릭 시 **Console 탭**에 나타나는 오류 메시지를 모두 확인합니다 3. 특히 `ReferenceError`, `TypeError` 등이 발생했었다면 해당 라인을 추적해 앞서 언급한 코드 수정이 제대로 이뤄졌는지 검증합니다. 수정 후에는 동일 시나리오에 오류가 재발생하지 않아야 합니다.
- **네트워크 요청 확인**: **Network 탭**에서 `/api/status` 및 `/api/items?state=ready` 호출이 이루어지는지 확인하세요. 새로고침 버튼 클릭 시 `/api/status` 요청이 보내지고 200(성공) 응답 후 화면 값이 바뀌는 지, 또는 토큰 해제 후 `/api/status` 요청이 보내져 401(Unauthorized)이 오는지 등을 살펴봅니다 3. 요청이 아예 보이지 않으면 프론트엔드에서 호출을 못한 것이므로 이벤트/스크립트 문제이고, 요청이 가지만 응답이 에러이면 백엔드/토큰 문제일 수 있습니다. 각 경우에 대해 앞서 수정한 로직(예: 헤더 포함 여부, fetch 오버라이드)을 재확인합니다.
- **스토리지 변화 확인**: **Application 탭**의 **Local Storage**를 열어 `ADMIN_TOKEN` 키의 변화를 모니터링합니다. 토큰 해제 버튼 클릭 전에 값이 존재하고, 클릭 직후 사라지는지, 그리고 페이지 reload 후에도 비어있는지 확인합니다. 만약 reload 후에도 다시 생긴다면, 수정한 코드 중 아직 토큰을 재설정하는 부분이 남아있는지 조사해야 합니다.
- **이벤트 핸들러 검사**: **Elements 탭**에서 버튼 요소를 검사하거나, Console에서 `getEventListeners(document.getElementById('btn-token-clear'))` 같은 명령을 실행해 **이벤트 리스너가 등록**되었는지 확인합니다. 새로고침 버튼도 동일하게 확인하여, 각각 클릭 시 연결된 함수(`clearAdminTokenJJ`, `refresh`)가 존재하는지 보세요. 등록이 보이면 바인딩 문제가 해결된 것이고, 여전히 없다면 스크립트 로드 순서나 ID 매칭 문제를 다시 확인해야 합니다.
- **스크립트 로드 확인**: Network 탭의 JS 파일 요청을 확인하여 **올바른 스크립트 파일이 로드되었는지** (상태 200, 최신 버전) 봅니다. 브라우저 캐시를 비우거나, 파일 이름 변경 등을 통해 반드시 최신 코드가 실행되는 상태에서 테스트해야 합니다.
- **UI 업데이트 확인**: KPI 수치나 아이템 목록 등의 **DOM 변화**를 실제로 눈으로 확인합니다. 새로고침 버튼 클릭 전후 KPI 숫자가 갱신되는지, `loadReady` 수정 후 목록에 아이템 행들이 출력되는지를 체크합니다. 필요하다면 DOM 변화를 추적하기 위해 Elements 탭에서 해당 영역을 미리 선택해두고 변화를 관찰하거나, Console에서 데이터 출력 후 DOM을 조회하여 반영된 값을 검증합니다.

사전 테스트 방안 (로컬 및 CI)

이번 문제를 해결한 후에는 **유사 문제가 재발하지 않도록** 미리 감지하는 체계를 갖추는 것이 중요합니다. 다음과 같은 테스트 전략을 제안합니다:

- **로컬 환경 재현 테스트**: 문제가 수정되면, **로컬 개발 환경**에서 해당 어드민 페이지를 띄워 **동일 시나리오를 재현**해 봅니다 9. 수동으로:
- `localStorage`에 `ADMIN_TOKEN` 값을 세팅 (또는 로그인 절차 수행)하여 로그인 상태로 진입합니다.

- 페이지가 로드되면 KPI 수치와 Ready 목록이 자동 갱신되는지 확인합니다. (loadReady 호출 수정 효과 확인)
- “새로고침” 버튼 클릭 -> Network 요청 발생 및 UI 업데이트 확인.
- “토큰 해제” 버튼 클릭 -> localStorage에서 토큰 제거 및 (로그아웃 처리) 동작 확인.

이 과정에서 앞서 언급한 DevTools 확인 방법을 활용하면 문제 없이 동작하는지 검증할 수 있습니다. 로컬에서 모든 기능이 정상이라면, 배포 환경과 차이를 만드는 요소(예: 정적 파일 누락, 환경변수 설정 등)는 없는지 점검합니다 ⁹ . - **통합(E2E) 테스트 시나리오 작성**: CI 파이프라인에 **엔드투엔드 테스트**를 추가하여, 브라우저 환경에서의 주요 UI 동작을 자동화합니다. 예를 들어 **Puppeteer**나 **Playwright**, **Cypress** 등을 활용해 헤드리스 브라우저로 admin 페이지를 열고 다음을 자동 검증할 수 있습니다: - 토큰을 미리 주입하여 로그인 상태로 진입한 후, 특정 UI 요소 (예: KPI 값 표시 영역) 내 텍스트가 `/api/status`의 데이터 응답과 일치하는지 확인. - “새로고침” 버튼을 클릭하는 동작을 스크립팅하여, 그 결과로 DOM의 KPI 값이 변경되는지 또는 특정 API 호출이 발생했는지 확인. - “토큰 해제” 버튼 클릭을 스크립팅하여, localStorage의 ADMIN_TOKEN 값이 제거되고, 페이지가 특정 로그인 페이지로 이동하거나 KPI 값이 초기화되는지 등을 확인.

이러한 E2E 테스트는 초기 구축에는 노력이 들지만, CI에서 **스모크 테스트를 확장**하는 효과가 있습니다 ¹⁰ . 즉, 단순히 `/api/status` 엔드포인트 호출 성공 여부만 확인하는 현재 스모크 테스트를 넘어, **사용자 상호작용을 포함한 시나리오까지 검증**하게 되어 향후 토큰 처리나 UI 업데이트 관련 문제가 생기면 배포 전에 감지할 수 있습니다 ¹⁰ . - **단위 테스트 및 코드 린트**: 가능하다면, fetch 오버라이드 함수나 clearAdminTokenJJ, refresh 등의 함수에 대해 **단위 테스트**를 작성해볼 수 있습니다. 예를 들어 Node 환경+JSDOM을 이용하여 window.localStorage와 fetch를 모킹하고, 함수 호출 시 기대 동작 (토큰 제거 후 localStorage 비었는지, fetch 호출 시 헤더가 설정되는지 등)을 확인하는 것입니다. 또한 ESLint 등의 정적 분석 도구를 활용하면, 정의되지 않은 변수를 참조하거나 API 사용을 잘못된 부분(예: Headers 객체에 객체리터럴 대입 등)을 미리 잡아낼 수 있으므로 적극 활용합니다. - **CI 환경에서의 브라우저 테스트**: 만약 브라우저 UI 테스트를 CI에 바로 포함시키기 부담된다면, 최소한 **배포 후 스크립트 로드 및 간단한 함수 호출**을 체크하는 스크립트를 추가할 수 있습니다. 예를 들어 CI 파이프라인에서 최신 배포 URL의 HTML을 받아와 `<script>` 경로를 파싱한 다음, 그 스크립트를 가져와 `clearAdminTokenJJ` 나 `refresh` 함수 정의 여부를 검사하는 간단한 체크를 수행할 수 있습니다. 또는 서버 사이드에서 `/api/status`를 토큰 없이 호출해 401이 오는지, 토큰을 넣어 호출해 200이 오는지 확인하는 절차도 포함하면 **백엔드 인증 동작과 프론트엔드 토큰 처리 로직의 정합성**을 지속적으로 검증할 수 있습니다 ⁷ .

결론

이번 어드민 페이지의 토큰 해제 및 새로고침 기능 장애는 **프론트엔드 측의 자바스크립트 로직 문제**에서 기인한 것으로 보입니다 ¹ . 이벤트 핸들러 미바인딩, fetch 인터셉터 버그, 토큰 삭제 후 흐름 처리 미흡 등이 복합적으로 작용해 **사용자 입력에 UI가 반응하지 않는** 상황이 발생했습니다. 근본 원인을 하나씩 진단한 결과를 바탕으로, 자바스크립트 코드의 몇 군데를 소폭 수정함으로써 문제를 해결할 수 있음을 제안드렸습니다. 특히 **브라우저 개발자 도구를 활용한 관찰**은 원인 파악과 검증에 매우 유용하므로, 실제 수정 적용 전에 반드시 콘솔 에러, 네트워크 트래픽 등을 점검해야 합니다 ¹¹ .

마지막으로, 수정 후에도 유사한 이슈가 재발하지 않도록 **로컬 및 CI 수준에서 테스트 체계**를 갖추는 것이 중요합니다 ¹⁰ . 자동화된 UI 시나리오 테스트나 강화된 스모크 테스트를 통해, 토큰 저장/삭제와 주요 버튼 동작이 기대대로 동작하는지 지속적으로 확인하면 향후 배포 시 신뢰성을 높일 수 있습니다. 이번 분석과 조치로 문제가 해결되길 바라며, 추가적인 이상 징후 발견 시 앞서 언급한 방법으로 원인을 파악하고 대응하시기를 권장드립니다.

¹ ² ³ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ 1022오류.txt

file:///file_00000000459462089ec951aa8fa7357c

⁴ ⁵ reactjs - Set default header for every fetch() request - Stack Overflow

<https://stackoverflow.com/questions/44820568/set-default-header-for-every-fetch-request>